

MODULE 07 · DOMAIN 1 - DATA INGESTION AND TRANSFORMATION

Programming, IaC and CI/CD

Lambda tuning, infrastructure as code, deployment pipelines, and the software engineering the exam expects of a data engineer.

DOMAIN 1 of 4 EXAM WEIGHT 34% MODULE 7 of 27

This is a data engineering exam that expects you to be a **software engineer too.**

The languages named in the exam guide

- ◆ **Python.** The default for Glue, Lambda, EMR and almost everything else.
- ◆ **SQL.** Arguably the most important, and the one most often underestimated.
- ◆ **Scala.** Native Spark. Appears in EMR and Glue job options.
- ◆ **Java, R, Bash and PowerShell** are named but rarely decide a question.
- ◆ You will not write code. You may be shown code and asked why it is slow.

ON THE EXAM

SQL is the one to be genuinely fluent in. Several questions describe a query and ask why it is expensive.

Distributed computing, in the parts you are asked to describe

- ◆ **Work is split across machines** that do not share memory.
- ◆ **Coordination costs something.** Moving data between them is the expensive part.
- ◆ **Partitioning** decides how work divides, and therefore how evenly it divides.
- ◆ **Failure is normal**, so tasks must be retryable and results deterministic.
- ◆ **Not everything parallelises.** A step depending on the previous row cannot split.

ON THE EXAM

Everything in module five about shuffles and skew is this slide applied to Spark.

Data structures, at the level asked

- ◆ **Hash structures** give constant-time lookup by key. DynamoDB thinks this way.
- ◆ **Trees** keep data sorted, which is what makes range queries efficient.
- ◆ **Graphs** model relationships. Amazon Neptune exists for exactly this.
- ◆ **A DAG is a graph** with direction and no cycles. Every orchestrator uses one.
- ◆ **Indexes are the practical version.** They trade write cost for read speed.

ON THE EXAM

You are asked to describe these, not implement them. The exam cares that you can match a structure to an access pattern.

Memory is the only performance dial. **CPU comes with it**, whether you wanted it or not.

The limits that shape architecture

LIMIT	VALUE
Timeout	15 minutes maximum. A hard limit, not raisable
Memory	128 MB to 10,240 MB, in 1 MB steps. CPU scales with it
Ephemeral storage at /tmp	512 MB by default, configurable to 10,240 MB
Deployment package	50 MB zipped, 250 MB unzipped. 10 GB as a container image
Layers	Five per function, sharing dependencies across functions
Concurrency	1,000 per account by default. A soft limit

ON THE EXAM

The 15 minute timeout is the one that eliminates Lambda from questions. Everything else has a workaround.

Attaching storage to a function

- ◆ **/tmp** is fast, local and ephemeral. It may or may not be empty next invocation.
- ◆ **Amazon EFS** mounts a shared file system into the function.
- ◆ **EFS is shared and persistent**, so several functions see the same files.
- ◆ **EFS requires the function to be in a VPC**, which adds cold start and complexity.
- ◆ **Clean up /tmp** before returning, or a reused environment runs out of space.

THE TIEBREAKER

Shared state between invocations means EFS. Scratch space for one invocation means /tmp.

Packaging a function and its dependencies

- ◆ **A zip archive** up to 50 MB uploaded, 250 MB unpacked with layers included.
- ◆ **Layers** share common dependencies across functions. Five per function.
- ◆ **A container image** raises the ceiling to 10 GB for heavy libraries.
- ◆ Container images also let you reuse an existing build and test toolchain.
- ◆ **Large packages slow cold starts.** Ship what you use, not everything installed.

ON THE EXAM

A data science library stack will not fit in a zip. That is the usual reason to reach for a container image.

Reserved or provisioned concurrency?

RESERVED CONCURRENCY

A ceiling

- Caps how many run at once
- Protects downstream dependencies
- Also guarantees that capacity is available
- No extra charge

PROVISIONED CONCURRENCY

A warm floor

- Keeps environments initialised and ready
- Removes cold start latency
- Charged for whether used or not
- For latency-sensitive workloads

THE TIEBREAKER

Reserved limits how many. Provisioned makes them fast to start. Questions confuse the two deliberately.

Cold starts, and what actually helps

- ◆ **A cold start** is the time to create an environment and run initialisation code.
- ◆ **Smaller packages start faster.** Remove dependencies you do not use.
- ◆ **Initialise clients outside the handler** so they are reused across invocations.
- ◆ **Attaching a VPC** adds to cold start. Only do it when the function needs private access.
- ◆ **SnapStart** restores from a snapshot instead of initialising. Java runtimes.

ON THE EXAM

Creating a database client inside the handler rebuilds it on every invocation. Outside, it is reused.

What triggers a Lambda function

EVENT SOURCE	BEHAVIOUR
S3 Event Notifications	One event per object. Asynchronous, retried twice by Lambda
Amazon EventBridge	Any AWS event or schedule. Asynchronous
Amazon SQS	Lambda polls the queue and deletes on success
Kinesis and DynamoDB Streams	Lambda polls per shard, preserving order
Amazon API Gateway	Synchronous. The caller waits for the response
Step Functions	Synchronous by default, within the state machine

ON THE EXAM

Asynchronous sources retry on your behalf. Synchronous ones return the error to the caller instead.

When an asynchronous invocation fails

- ◆ **Lambda retries twice** automatically, with delays, then discards.
- ◆ **An on-failure destination** sends the failed event to SQS, SNS or EventBridge.
- ◆ **Destinations carry the response**, not just the event, so you know why it failed.
- ◆ **A dead-letter queue** is the older mechanism and keeps only the event.
- ◆ **With neither configured**, a failed asynchronous invocation vanishes silently.

THE TIEBREAKER

Destinations are the modern option and record the failure reason. A DLQ keeps the payload and not the cause.

An AWS Lambda function transforms JSON files from Amazon S3 and writes Parquet. It is configured with 512 megabytes of memory and takes about forty seconds per file, mostly spent in the Parquet conversion, which is CPU bound.

The team processes about eighty thousand files a month and wants to reduce the Lambda bill. Memory usage reported in CloudWatch peaks at around 180 megabytes, well under the configured amount.

What should the data engineer do?

- ☐ A Reduce the memory from 512 megabytes to 256, since only 180 are used
- ☐ B Increase the memory, measure the change in duration, and settle on the cheapest total
- ☐ C Enable provisioned concurrency so the functions do not pay cold start time
- ☐ D Increase the ephemeral storage allocation so the conversion has more room

◆ PAUSE AND DECIDE

What should the data engineer do?

- A

 Halve the memory
The intuitive answer, and it halves the CPU too. A CPU-bound function takes roughly twice as long, so the bill barely moves and may rise.
- B

 Raise memory, measure, pick the cheapest total
More memory means more CPU. A CPU-bound function finishes proportionally sooner, and duration times rate often falls.
- C

 Provisioned concurrency
Removes cold start latency at a continuous charge. The work takes forty seconds; cold start is not the cost here.
- D

 More ephemeral storage
Storage was never the constraint. Memory usage is low and nothing reported a disk problem.

THE SIGNAL

"CPU bound" plus "memory barely used" is the exam pointing at the memory-equals-CPU relationship.

Where else code runs

- ◆ **AWS Batch** schedules containerised jobs onto managed compute. No 15 minute limit.
- ◆ Good for long-running, embarrassingly parallel work that is not Spark.
- ◆ **Amazon ECS and EKS** for containers you want running as a service.
- ◆ **Glue Python shell** for a small script that needs Glue's scheduling and logging.
- ◆ Reach for Batch when work is too long for Lambda and too simple for Spark.

ON THE EXAM

AWS Batch is in the in-scope service list and is frequently forgotten. It fills the gap above Lambda.

If a pipeline exists only because someone clicked, it cannot be **rebuilt, reviewed or trusted.**

CloudFormation or the CDK?

AWS CLOUDFORMATION

Declarative templates

- YAML or JSON describing resources
- No programming language to learn
- Verbose for repetitive resources
- The engine underneath everything

AWS CDK

Code that generates templates

- Python, TypeScript, Java and others
- Loops and functions remove repetition
- Synthesises to CloudFormation anyway
- Requires the language and the CDK

THE TIEBREAKER

The CDK does not replace CloudFormation. It generates it, so everything CloudFormation does still applies.

CloudFormation mechanics worth knowing

- ◆ **A stack** is a set of resources managed together and deleted together.
- ◆ **Change sets** show what a deployment will do before it does it.
- ◆ **Rollback** returns the stack to its previous state when a deployment fails.
- ◆ **Drift detection** reports resources changed outside the template.
- ◆ **StackSets** deploy the same stack across many accounts and Regions.

ON THE EXAM

Change sets are the review step. They turn "apply and hope" into "read the diff, then apply".

AWS SAM, and where it fits

- ◆ **An extension of CloudFormation** with shorthand for serverless resources.
- ◆ **A few lines** define a function, its role, its trigger and its permissions.
- ◆ **Local testing.** Invoke functions and emulate events on your machine.
- ◆ **Named explicitly in the exam guide** for packaging and deploying serverless pipelines.
- ◆ Serverless focused. Not the tool for an EMR cluster or a VPC.

ON THE EXAM

SAM is CloudFormation with a serverless shorthand. A SAM template is a CloudFormation template.

Where configuration lives

STORE	USE IT FOR
Systems Manager Parameter Store	Non-secret configuration. Bucket names, thresholds, feature flags
AWS Secrets Manager	Secrets. Database passwords, third-party API keys
Environment variables	Small, non-secret values known at deploy time
Job parameters	Per-run values such as the date being processed

ON THE EXAM

Secrets Manager rotates automatically and costs per secret. Parameter Store does not rotate and has a free tier.

A data platform team manages sixty AWS Glue jobs across three environments. Each environment differs only in bucket names, database names and worker counts. Jobs are currently created by hand in the console, and configuration drift between environments has twice caused a production failure that did not appear in test.

The team wants environments to be provably identical apart from their parameters, and wants changes reviewed before they are applied.

What should the team implement?

- ☐ A Document the console steps thoroughly and have a second engineer check each change
- ☐ B Define the jobs with the AWS CDK, parameterised per environment, and deploy through a pipeline
- ☐ C Copy the production Glue jobs into the other environments using the AWS CLI after each change
- ☐ D Use AWS Config rules to detect when environments diverge

◆ PAUSE AND DECIDE

What should the team implement?

- A

 Documentation and a second pair of eyes
A process control over a manual step. It reduces mistakes without removing the cause, and drift returns the first busy week.
- B

 CDK, parameterised, deployed through a pipeline
Sixty near-identical jobs become a loop, environments differ only by parameter, and changes go through review before deployment.
- C

 Copy with the CLI after each change
Scripted drift. It still depends on somebody remembering to run it, and it copies mistakes faithfully.
- D

 Config rules to detect divergence
Tells you the environments have drifted. It does not stop them drifting or fix them when they have.

THE SIGNAL

"Provably identical apart from parameters" is the phrase that demands a definition, not a process.

Version control for pipeline code

- ◆ **Everything in Git.** Job scripts, templates, SQL, DAG definitions, configuration.
- ◆ **Branch, review, merge** so no change reaches production unreviewed.
- ◆ **AWS CodeCommit was removed** from the exam's in-scope services in December 2025.
- ◆ **CodePipeline, CodeBuild and CodeDeploy remain in scope**, working with any Git provider.
- ◆ **Glue jobs support source control integration** directly.

ON THE EXAM

Availability warning: CodeCommit closed to new customers in 2024 and left the exam's in-scope list in 2025.

How pipeline code reaches production

01

Commit

A change is pushed to a branch and reviewed.

02

Build

CodeBuild runs tests and packages artefacts.

03

Deploy to test

CloudFormation or CDK applies the change.

04

Validate

Run the pipeline against known data and check the output.

05

Promote

The same artefact is deployed to production.

ON THE EXAM

The same artefact moves through environments. Rebuilding per environment is how test and production diverge.

Deploying a change safely

- ◆ **Versions** are immutable snapshots of a function. **Aliases** point at a version.
- ◆ **Move the alias** to release. Move it back to roll back, in seconds.
- ◆ **CodeDeploy shifts traffic gradually**, canary or linear, watching alarms.
- ◆ **Automatic rollback** when an alarm fires during the shift.
- ◆ **Batch pipelines are harder to canary**, because one nightly run is all or nothing.

ON THE EXAM

Gradual traffic shifting works for request-driven functions. A nightly job needs validation before promotion instead.

Testing something that processes data

- ◆ **Unit tests** on transformation functions, with small hand-built inputs.
- ◆ **Integration tests** that run the job against a known sample dataset.
- ◆ **Assert on the output**, not only that the job exited successfully.
- ◆ **Data quality checks in the pipeline** catch what tests cannot predict.
- ◆ **A job that succeeds** tells you nothing about whether the numbers are right.

ON THE EXAM

Testing pipeline code and validating pipeline data are different activities. You need both.

A team deploys AWS Glue job scripts by uploading them to Amazon S3 and updating the job definition manually. A recent change was applied to production before it had been run in test, and it silently produced incorrect aggregates for two days before anyone noticed.

The team wants changes to be tested before reaching production, and wants to detect incorrect output rather than only failed jobs.

Which combination best addresses both requirements?

- ☐ A A CodePipeline that builds and tests in a test environment, promotes the same artefact to production, plus data quality checks in the job
- ☐ B Add retries and a longer timeout to the Glue job so transient failures do not reach production
- ☐ C Require a second engineer to approve each manual upload to Amazon S3
- ☐ D Enable Glue job bookmarks so incorrect data is not reprocessed

◆ PAUSE AND DECIDE

Which combination best addresses both requirements?

- A

 CodePipeline with promotion, plus in-job data quality checks
The pipeline stops untested code reaching production; the quality checks catch output that is wrong rather than absent.
- B

 Retries and a longer timeout
Handles transient failure. Nothing failed here. The job succeeded and produced wrong numbers.
- C

 Manual approval of uploads
A human gate on a manual process. It catches some mistakes and does not test anything.
- D

 Job bookmarks
Controls which source data is read. It has no bearing on whether the transformation is correct.

THE SIGNAL

Two requirements, two mechanisms. Deployment discipline solves the first; data validation solves the second.

Optimising code that already works

- ◆ **Filter and project early.** Reading less beats processing faster.
- ◆ **Reuse connections and clients** across invocations or partitions.
- ◆ **Batch external calls** instead of one call per record.
- ◆ **Row-by-row processing** of a large dataset is the classic slow pattern.
- ◆ **Measure before optimising.** The bottleneck is rarely where it feels.

ON THE EXAM

Almost every optimisation on this exam reduces work rather than speeding it up.
Module five said the same about cost.

Calling AWS from code

- ◆ **The SDK** is how a job talks to AWS. Boto3 in Python is the common one.
- ◆ **Credentials come from the execution role**, never from keys in the code.
- ◆ **SDKs retry automatically**, with backoff, for throttling and transient errors.
- ◆ **That automatic retry hides problems**, so throttling can be invisible in your logs.
- ◆ **Paginate**. List operations return a page at a time, not everything.

ON THE EXAM

Hard-coded access keys in a job script are always the wrong answer. The execution role is always the right one.

Making a job observable

- ◆ **Log structured records**, not free text. Machines have to read them later.
- ◆ **Include a correlation identifier** so one run can be traced across services.
- ◆ **Emit custom metrics** for things that matter, like rows processed or rejected.
- ◆ **Log the decision, not just the error.** Why a record was skipped is the useful part.
- ◆ **Logging every record** costs more than the processing and buries the signal.

ON THE EXAM

Rows in and rows out, emitted as metrics, catches silent data loss that no error ever reports.

A nightly AWS Glue job reads from a partner API using an access key and secret stored in the job's script parameters. The job lists objects in an Amazon S3 bucket, processes them, and writes results.

Two problems have emerged. The partner rotated the credentials and the job failed until someone edited the parameters. Separately, the job has been quietly processing only the first thousand objects in a bucket that now holds forty thousand.

Which combination fixes both problems?

- ☐ A Store the partner credentials in AWS Secrets Manager and paginate the S3 list operation
- ☐ B Increase the Glue job timeout and add more DPU's so all objects are processed
- ☐ C Move the credentials into an environment variable and add a retry to the list call
- ☐ D Attach an IAM role with S3 access and enable job bookmarks

◆ PAUSE AND DECIDE

Which combination fixes both problems?

- A

 Secrets Manager plus pagination
Secrets Manager holds and rotates the third-party credential; pagination is why only a thousand objects were seen.
- B

 Longer timeout and more DPUs
Capacity for a problem that is not capacity. The job is not running out of time, it is asking for one page.
- C

 Environment variable plus retry
An environment variable is still a stored secret and still needs manual rotation. A retry returns the same first page.
- D

 IAM role and bookmarks
An IAM role is right for AWS access, and this credential belongs to a partner, so a role cannot represent it.

THE SIGNAL

The thousand-object number is planted. It is the default page size, not a coincidence.

Amazon Q, and where it appears

- ◆ **Amazon Q is on the in-scope service list** under Developer Tools.
- ◆ Generates and explains code, including SQL and transformation logic.
- ◆ **Glue and Redshift both surface it** for authoring jobs and queries.
- ◆ Added to the exam scope in the December 2025 revision.
- ◆ Assistance with authoring, not a runtime component of a pipeline.

ON THE EXAM

Newly in scope. Expect it as an authoring-productivity option, not as something that processes data.

Reading a programming question quickly

- ◆ **Is it CPU bound or waiting?** That decides whether more Lambda memory helps.
- ◆ **Could it exceed fifteen minutes?** If so, Lambda is eliminated.
- ◆ **Are credentials in the code?** Roles for AWS, Secrets Manager for everything else.
- ◆ **Is the problem drift or repetition?** Both point at infrastructure as code.
- ◆ **Did it fail, or did it succeed wrongly?** Those need completely different answers.

ON THE EXAM

Domain 1 complete. Next: Domain 2 begins with Amazon S3 as a data lake.